Universidad Iberoamericana León

Ing. Alejandro Montes Moreno
Desarrollo de Aplicaciones y Servicios Virtuales

Práctica guiada: implementación del primer módulo

Codificación, revisión y pruebas con apoyo de IA · Entorno Antigravity

Proyecto	**COTIZA-ELEV** — Sistema Web de Cotizaciones para Elevadores (ELEVESE)	**Fecha**	3 de noviembre de 2026
Integrantes	• Orlando Villalobos Gutiérrez (Arquitectura y Backend) • Diego Emilio Alférez Vallejo (QA, Persistencia y Frontend)	**Repositorio**	github.com/DiegoAlferez-AI/COTIZA-ELEV

Paso 1. Seleccionar el módulo y definir la orden de implementación

Campo	Respuesta del equipo
RF / HU / CA vinculados	**RF-28** (Cálculo automático de subtotales, IVA y total) / **HU-16** (Cálculo comercial) / Criterios: **CA-16.1** (Normal), **CA-16.2** (Límite centavos), **CA-16.3** (Error).
Nombre y propósito del módulo	**M07: Motor de Cálculo Comercial (`CalculoMotorService`)**. Encapsular la lógica matemática pura del negocio para calcular importes de partidas, subtotal, cálculo de IVA (16%) y total formal a dos decimales con redondeo estándar.
Entradas y datos que recibe	Arreglo `$partidas` con `cantidad` (> 0), `precio_unitario` (≥ 0), `descripcion` (string opcional). Parámetro opcional: `float $tasaIva = 0.16`.
Salidas y mensajes esperados	Estructura: `subtotal`, `tasa_iva`, `monto_iva`, `total` y `partidas` desglosadas con importe por renglón.
Validaciones y casos de error	1. Rechazar cantidades ≤ 0. 2. Rechazar precios unitarios negativos. 3. Lanzar `InvalidArgumentException` ante datos no numéricos. 4. Si la lista está vacía, retornar ceros en subtotal/IVA/total.
Dependencias con otros módulos	Ninguna. Es una clase de dominio puro desacoplada de la base de datos (M04/M06) y de las interfaces web.
Archivos que puede crear o modificar	`src/Services/CalculoMotorService.php` y `tests/Unit/CalculoMotorServiceTest.php`.
Restricciones: qué no debe cambiar	No conectarse a MySQL (PDO), no usar `$_SESSION`, no renderizar HTML ni alterar estados fuera de memoria.

Pruebas mínimas requeridas	Normal (CA-16.1), Límite con redondeo de centavos (CA-16.2) y Errores de validación de precio/cantidad (CA-16.3).

Pregunta de control: ¿la implementación propuesta conserva el alcance y la separación de responsabilidades previstas? Justifiquen.
Sí, respeta íntegramente la arquitectura modular por capas aprobada. Al mantener el motor de cálculo dentro de `src/Services/CalculoMotorService.php` como un servicio puro sin llamadas a MySQL ni vistas HTML, se previene el antipatrón de "God Object" en el Cotizador (M06) y se garantiza que las fórmulas comerciales sean reutilizables de forma idéntica en pantallas interactivas, base de datos y reportes PDF.

Registro de la decisión del equipo sobre la orden: `Aceptada` — Cumple con el alcance delimitado del Hito 1 (Tareas T-04 y T-05) y cuenta con criterios matemáticos verificables.

Paso 2. Análisis técnico antes de codificar

Aspecto a revisar	Resultado / notas del equipo
El plan respeta el alcance y el diseño modular	Sí, acotado estrictamente a la lógica pura del motor M07 sin tocar base de datos ni vistas.
Los archivos propuestos son razonables	Sí, se crean exclusivamente `src/Services/CalculoMotorService.php` y la suite en `tests/`.
Se identificaron riesgos o dudas relevantes	Riesgo de discrepancias de centavos mitigado aplicando redondeo `round($val, 2, PHP_ROUND_HALF_UP)` sobre subtotal acumulado.
Se requiere ajustar la orden de implementación	No, la orden técnica es clara, completa y viable.

Propuesta del asistente	Decisión del equipo	Justificación
Implementar `CalculoMotorService` con PHP 8.2 estricto y suite de pruebas unitarias automatizadas ejecutables en consola CLI.	**Aceptada**	Permite verificar la lógica financiera con evidencia de ejecución real antes de tocar la base de datos o interfaces.

Paso 3. Autorizar e implementar el módulo

Elemento	Registro
Archivos efectivamente creados	`src/Services/CalculoMotorService.php`, `tests/Unit/CalculoMotorServiceTest.php`, `tests/run_tests.php`, `README.md` y `.gitignore`.
Funciones / clases implementadas	Clase `CalculoMotorService` con métodos: • `calcularImportePartida(float|int $cantidad, float|int $precioUnitario): float` • `calcularTotales(array $partidas, float $tasaIva = 0.16): array`
Validaciones incorporadas	Rechazo de cantidad ≤ 0, rechazo de precio unitario < 0, validación de tipos numéricos y parámetros de IVA no negativos.
Mensajes de error o retorno	`"La cantidad debe ser mayor a 0. Recibido: {cantidad}"` `"El precio unitario no puede ser negativo. Recibido: {precio}"`
Cambios que el equipo decidió ajustar	Uso de `PHP_ROUND_HALF_UP` a 2 decimales en el cálculo de cada importe y sobre el IVA del subtotal para garantizar consistencia con los libros contables.

Paso 4. Diseñar, ejecutar y revisar pruebas

Caso	Datos de entrada	Resultado esperado	Resultado real obtenido	Estado
Normal / CA-16.1	2 piezas a $1,000.00 c/u + 1 mano de obra a $500.00 (IVA 16%).	Subtotal: $2,500.00 IVA: $400.00 Total: $2,900.00	Subtotal: $2500.00 IVA: $400.00 Total: $2900.00	**Pasa [OK]**
Límite / CA-16.2	3 refacciones a $333.33 c/u ($999.99).	Subtotal: $999.99 IVA: $160.00 exacto Total: $1,159.99	Subtotal: $999.99 IVA: $160.00 Total: $1159.99	**Pasa [OK]**
Error / CA-16.3 (A)	Precio unitario negativo (-500.00).	Lanzar `InvalidArgumentException`.	Excepción capturada: *"El precio unitario no puede ser negativo. Recibido: -500"*	**Pasa [OK]**
Error / CA-16.3 (B)	Cantidad en cero (0).	Lanzar `InvalidArgumentException`.	Excepción capturada: *"La cantidad debe ser mayor a 0. Recibido: 0"*	**Pasa [OK]**

Elemento de evidencia	Registro
Comando de ejecución	`C:\xampp\php\php.exe tests\run_tests.php`

Salida observada en consola	`============================================================`
	` EJECUCIÓN DE PRUEBAS UNITARIAS: CalculoMotorService (M07)`
	` Entorno: PHP 8.2.12 en Windows`
	`============================================================`
	`[PRUEBA 1 - NORMAL / CA-16.1] Cálculo estándar: PASA [OK]`
	`[PRUEBA 2 - LÍMITE / CA-16.2] Precisión de centavos: PASA [OK]`
	`[PRUEBA 3 - ERROR / CA-16.3] Precio negativo: PASA [OK]`
	`[PRUEBA 4 - ERROR / CA-16.3] Cantidad en cero: PASA [OK]`
	`------------------------------------------------------------`
	`RESULTADO FINAL: 4 de 4 pruebas pasaron exitosamente.`
	`ESTADO: APROBADO [100% SUCCESS]`
	`============================================================`

Explicación por integrante	• **Orlando Villalobos:** Métodos puros, aislamiento matemático y precisión de redondeo financiero. • **Diego Alférez:** Suite de pruebas unitarias, cobertura de casos normal, límite y error, y aserción de centavos.

Paso 5. Crear commit, hacer push y verificar GitHub

Evidencia	Dato
URL del repositorio / rama	https://github.com/DiegoAlferez-AI/COTIZA-ELEV / rama `main`
Hash o referencia del commit	`9fb2b49d22bea5329fe4bbb869673fa8c95607b6` (corto: `9fb2b49`)
Rama local y remota en GitHub	`main -> origin/main` (Sincronizado al 100%)
Resultado del push	`To https://github.com/DiegoAlferez-AI/COTIZA-ELEV.git * [new branch] main -> main (Exitoso)`
URL del commit visible en GitHub	github.com/DiegoAlferez-AI/COTIZA-ELEV/commit/9fb2b49
RF y módulo implementado	**RF-28** (Cálculo de cotizaciones) / **M07: Motor de Cálculo Comercial**
Pruebas que pasan y pendientes	Pasan: 4 pruebas (CA-16.1, CA-16.2, CA-16.3A, CA-16.3B). Pendientes: Ninguna en M07.
Ubicación de la evidencia	Archivo ejecutable `tests/run_tests.php` ejecutado en terminal con PHP 8.2.
Siguiente tarea o dependencia	T-06 (Servicio de autenticación RBAC M01) y T-01 (Script DDL MySQL InnoDB).

Paso 6. Revisión final y decisión del equipo

Registro de la decisión del equipo:
*El módulo M07 (`CalculoMotorService`) queda **Aceptado formalmente**. Cumple íntegramente con los requerimientos RF-28, HU-16 y criterios de aceptación CA-16.1 al CA-16.3. Conserva como pendiente su vinculación con el Cotizador web (M06) en el Hito 3 del proyecto.*

Checklist de Definición de Terminado (DoD)	Estado
El módulo cumple el RF y conserva las entradas y salidas previstas.	**[X] Sí** / [ ] No
Las validaciones y errores definidos están contemplados.	**[X] Sí** / [ ] No
Se ejecutaron pruebas normal, límite y error con resultados anotados.	**[X] Sí** / [ ] No
El código se entiende y cada integrante puede explicar una parte.	**[X] Sí** / [ ] No
Cada commit de código tiene push y está publicado en GitHub.	**[X] Sí** / [ ] No
Se actualizó trazabilidad y bitácora de IA.	**[X] Sí** / [ ] No

Entrega y reflexión individual

Integrante	¿Qué parte del módulo puede explicar?	¿Qué aprendió al comparar diseño y ejecución?
Orlando Villalobos Gutiérrez	La estructura de la clase `CalculoMotorService`, el aislamiento	Que aislar la lógica matemática en un módulo desacoplado permite verificar reglas comerciales

	del dominio puro y el redondeo estricto con `round($subtotal * $iva, 2, PHP_ROUND_HALF_UP)`.	en milisegundos sin requerir servidores web ni bases de datos activas.
Diego Emilio Alférez Vallejo	La suite de pruebas en `tests/run_tests.php`, los casos de borde de centavos (3 refacciones a $333.33) y la captura de excepciones `InvalidArgumentException`.	Que definir con rigor los criterios de aceptación (Normal, Límite, Error) antes de codificar permite automatizar pruebas con evidencia real y sin ambigüedades.